

Importation de pytorch / l'encoder tiktoken

```
!pip install tiktoken

import torch
import torch.nn as nn
import torch.nn.functional as F
import tiktoken
import os
```

[Afficher la sortie masquée](#)

Parametre global du modele

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(device)
batch_size = 16
block_size = 32
n_embd = 128
n_head = 4
n_layer = 2
dropout = 0.2
lr = 3e-4
epochs = 1000

eval_iters = 200
eval_interval = 500

torch.manual_seed(1337)

cpu
<torch._C.Generator at 0x7a7230e38ff0>
```

Telechargement du dataset et encodage grace a tiktoken et separation en train et test

```
!wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt -O shakespeare.t
with open("shakespeare.txt", "r", encoding="utf-8") as f:    text = f.read()

enc = tiktoken.get_encoding("gpt2")
vocab_size = enc.n_vocab
tokens = enc.encode(text)
data = torch.tensor(tokens, dtype=torch.long)

n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]

--2025-12-02 03:37:57--  https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.tx
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.111.
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1115394 (1.1M) [text/plain]
Saving to: 'shakespeare.txt'

shakespeare.txt    100%[=====>]    1.06M  --.-KB/s    in 0.03s

2025-12-02 03:37:57 (32.8 MB/s) - 'shakespeare.txt' saved [1115394/1115394]
```

Visiulation de comment est associe chaque token avec tiktoken

```
print(text[:13])
print(train_data[:2])
print(train_data[:1], enc.decode_single_token_bytes(train_data[:1]))
print(train_data[1:2], enc.decode_single_token_bytes(train_data[1:2]))

First Citizen
tensor([ 5962, 22307])
tensor([5962]) b'First'
tensor([22307]) b' Citizen'
```

Model : le modele s'inspire de celui dans le papier : Attention Is All You Need Ici on ne vas garder que le decodeur car on na pas besoin de l'encoder car l'objectif est de faire un modele de prediction du prochain token pas de traduire un text. De plus nous allons

l'améliorer avec les derniers papiers de recherche sortis. De plus ici notre modèle n'aura plus besoin du deuxième bloc d'attention.

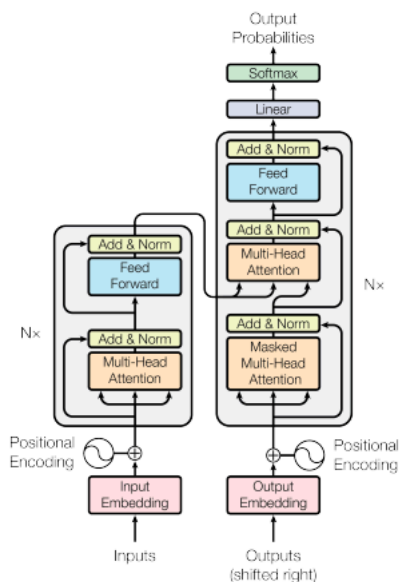
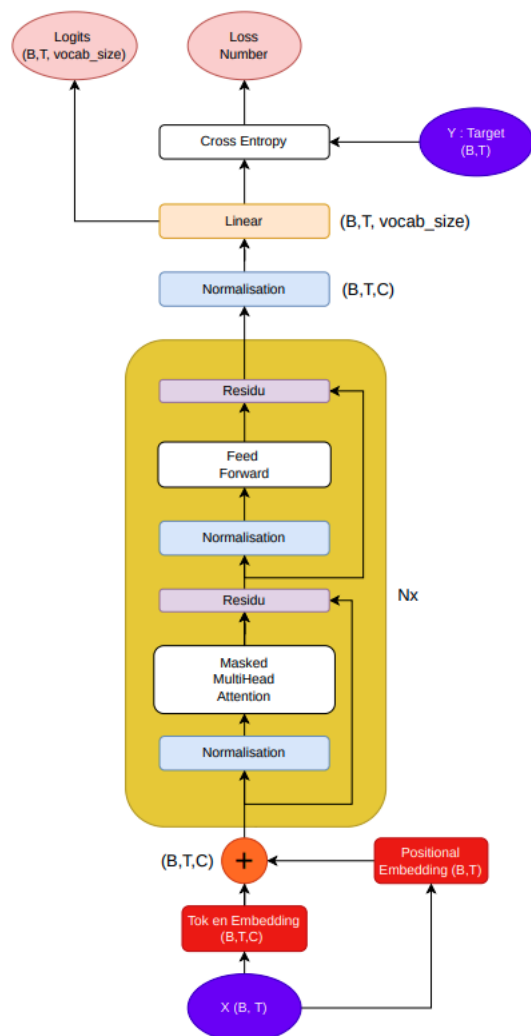
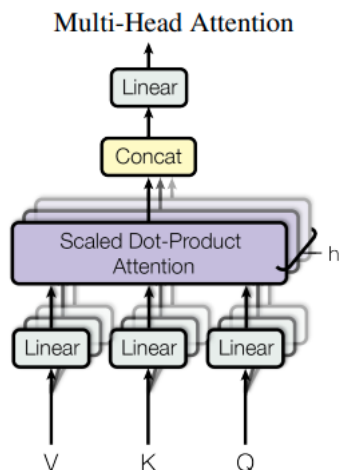


Figure 1: The Transformer - model architecture.

Voici le modèle que l'on va suivre. Nous coderons chaque sous-parti.

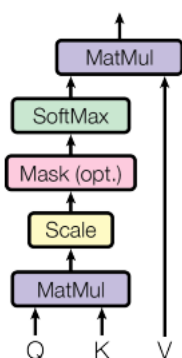


Commençons par le bloc de multihead attention on va s'inspirer de celui dans le papier : Attention Is All You Need Cependant on voit qu'il nous manque le bloque Scaled Dot-Product ainsi on va commencer par coder celui-ci



Ainsi on va suivre le modèle qu'on retrouve dans le papier Attention Is All You Need pour le bloc Scaled Dot-Product Attention. Et on va rajouter un Dropout avant le dernier MatMul. Il sert de mécanisme de régulation, il permet que le modèle se généralise.

Scaled Dot-Product Attention



```
class attention(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.head_size = head_size
        self.drop = nn.Dropout(dropout)
        self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

    def forward(self, Q,K,V):
        B,T,C = Q.shape
        wei = Q @ K.transpose(-2, -1) / self.head_size**0.5 #MatMul + Scale
        wei = wei.masked_fill(self.tril[:T,:T] == 0, float('-inf')) #Mask
        wei = F.softmax(wei, dim = -1) #SoftMax
        wei = self.drop(wei) #DropOut
        out = wei @ V #MatMul
        return out #(B,T,Head_size)
```

Pour le Block de Multi Head Attention on rajoute aussi un Dropout avant la linearisation final pour mieux generaliser

```
class MultiHeadAttention(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.Q = nn.Linear(n_embd, head_size, bias = False)
        self.K = nn.Linear(n_embd, head_size, bias = False)
        self.V = nn.Linear(n_embd, head_size, bias = False)
        self.Heads = nn.ModuleList([attention(head_size) for i in range (n_head)])
        self.lin = nn.Linear(head_size*n_head, n_embd)
        self.drop = nn.Dropout(dropout)

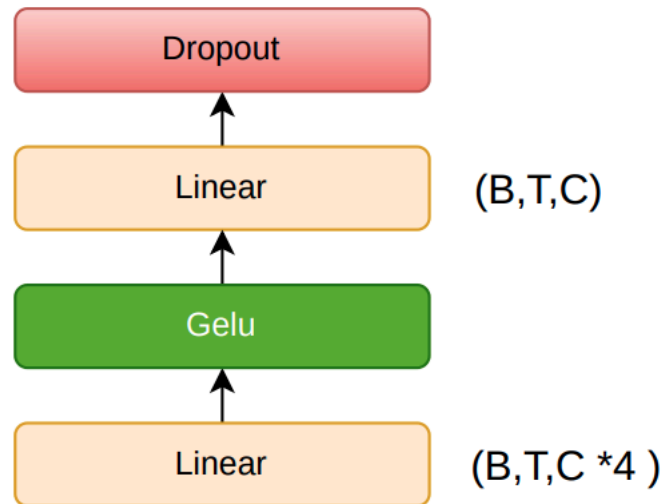
    def forward(self, x):
        Q = self.Q(x) #linear
        K = self.K(x) #linear
```

```

V = self.V(x) #linear
out = torch.cat([head(Q,K,V) for head in self.Heads], dim = -1) #Multi Attention *H + Concat
out = self.drop(out) #Dropout
out = self.lin(out) #Linear
return out

```

Maintenant il faut coder le block feed forward ici aussi on rajoute un dropout et de plus on augmente la dimension de 4 au début pour augmenter la capacité d'apprendre



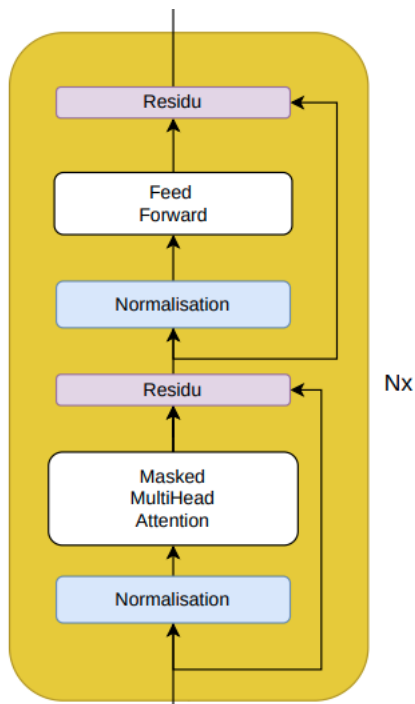
```

class Feed_Forward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.FN = nn.Sequential(
            nn.Linear(n_embd, n_embd*4), #(B, T, C*4)
            nn.GELU(),
            nn.Linear(n_embd*4, n_embd), #(B, T, C)
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.FN(x) #Feed Forward

```

Il nous reste plus qu'à assembler le tout dans le Block qu'on va multiplier n fois



```

class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.MHA = MultiHeadAttention(head_size)
        self.FNN = Feed_Forward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.MHA(self.ln1(x))
        x = x + self.FNN(self.ln2(x))
        return x
  
```

Et enfin on peut faire le model au total

```

class Camille0(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embedding_tok = nn.Embedding(vocab_size, n_embd)
        self.embedding_pos = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head) for i in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        B, T = x.shape
        token_emb = self.embedding_tok(x) # (B,T,C)
        position = self.embedding_pos(torch.arange(T, device=device).clamp(max=block_size-1)) # (T,C)
        x = token_emb + position # (B,T,C)
        x = self.drop(x)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x)
        logits = self.lm_head(x)

        return logits
  
```

Il faut cree une fonction qui vas nous retourner notre x et notre target de batch aleatoire

```

def get_batch():
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y
  
```

Enfin il faut initialiser notre model et cree notre optimiser pour l'entrainement ici on prend AdamW un des plus utiliser

```
model = Camille0(vocab_size).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

Et on peu entrainer nos donner sur notre modele

```
for epoch in range(epochs):
    xb, yb = get_batch()
    logits = model(xb)
    B, T, C = logits.shape

    logit = logits.view(B*T, C)
    target = yb.view(B*T)

    # entrainement
    loss = F.cross_entropy(logit, target)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
    if epoch % 100 == 0 or epoch == epochs-1:
        print(epoch, loss)

0 tensor(6.0360, grad_fn=<NllLossBackward0>)
100 tensor(5.7371, grad_fn=<NllLossBackward0>)
200 tensor(5.2195, grad_fn=<NllLossBackward0>)
300 tensor(5.1478, grad_fn=<NllLossBackward0>)
400 tensor(4.8060, grad_fn=<NllLossBackward0>)
500 tensor(4.5905, grad_fn=<NllLossBackward0>)
600 tensor(4.7437, grad_fn=<NllLossBackward0>)
700 tensor(4.6628, grad_fn=<NllLossBackward0>)
800 tensor(4.8720, grad_fn=<NllLossBackward0>)
900 tensor(4.3548, grad_fn=<NllLossBackward0>)
999 tensor(4.7530, grad_fn=<NllLossBackward0>)
```

Ici on remarque bien une decroissance de la loss ainsi on peu en conclure que notre modele apprend